

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño. Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Mauricio León Osnovikoff Delgado

PARALELO: A

SEMESTRE: Quinto

ACTIVIDAD 1: Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL – AGOSTO 2026



Actividad # 1

Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

1. Análisis del Problema

El presente análisis tiene como objetivo identificar los elementos principales necesarios para el diseño de un sistema de gestión de incidentes orientado a un entorno de Data Center. Para ello, se definen los actores involucrados, así como los requerimientos funcionales y no funcionales.

2. Identificación de Actores del Sistema

Para el diseño del sistema de gestión de incidentes orientado a un entorno de Data Center, se han identificado los siguientes actores principales que interactúan con la plataforma. Estos representan los diferentes roles involucrados en la gestión, atención y supervisión de incidentes técnicos.

Actores principales

Usuario Final: Es la persona que detecta y reporta un incidente en la infraestructura. Su interacción principal consiste en registrar tickets y consultar su estado.

Técnico de Soporte: Es el responsable de atender los incidentes asignados. Se encarga de diagnosticar, resolver problemas y actualizar el estado de los tickets.

Administrador de Sistemas: Gestiona la configuración general del sistema, administra usuarios y técnicos, y supervisa el correcto funcionamiento de la plataforma.

Supervisor de Soporte: Este actor tiene un rol de control y supervisión. Se encarga de monitorear el estado de los tickets, evaluar el desempeño del equipo técnico y gestionar el escalamiento de incidentes críticos o no resueltos en el tiempo esperado.

Sistema de Monitoreo: Corresponde a un sistema externo automatizado que detecta fallos en la infraestructura y genera tickets automáticamente sin intervención humana.

3. Requerimientos Funcionales

Los requerimientos funcionales describen las acciones específicas que el sistema debe ser capaz de realizar para cumplir con su propósito.

Crear ticket: Permite la creación de tickets de incidentes por parte de los usuarios.

Categorizar fallo: Clasifica los incidentes según su tipo (Hardware, Red, Software).

Asignar técnico: Asigna técnicos de soporte a los tickets generados.

Notificación automática: El sistema deberá notificar a los técnicos cuando se les asigne un nuevo incidente.



Actualizar estado del ticket: Permite cambiar el estado del incidente (Abierto, En proceso, Resuelto, Cerrado).

Consultar estado del ticket: Permite a los usuarios visualizar el estado actual de sus incidentes registrados.

Gestionar usuarios y técnicos: Permite al administrador crear, modificar o eliminar usuarios y técnicos dentro del sistema, así como asignar roles y permisos según corresponda.

Creación automática de tickets: Permite al sistema generar tickets de manera automática a partir de alertas provenientes de herramientas de monitoreo, sin intervención del usuario final.

Escalamiento de tickets: El sistema permitirá escalar incidentes a un supervisor en caso de no ser resueltos dentro de un tiempo determinado o si presentan alta criticidad.

4. Requerimientos No Funcionales

Los requerimientos no funcionales definen las características de calidad que debe cumplir el sistema.

Escalabilidad: El sistema debe soportar un crecimiento en el número de usuarios y tickets sin afectar su rendimiento.

Mantenibilidad: El sistema debe estar diseñado de manera modular para facilitar futuras mejoras y correcciones.

Trazabilidad: El sistema debe permitir el seguimiento de cambios y versiones mediante el uso de herramientas como Gitflow.

Seguridad: El sistema debe garantizar el acceso controlado mediante roles, asegurando que cada usuario solo pueda realizar acciones permitidas según su perfil.

Rendimiento: El sistema debe responder de manera eficiente, incluso bajo condiciones de alta carga de trabajo.

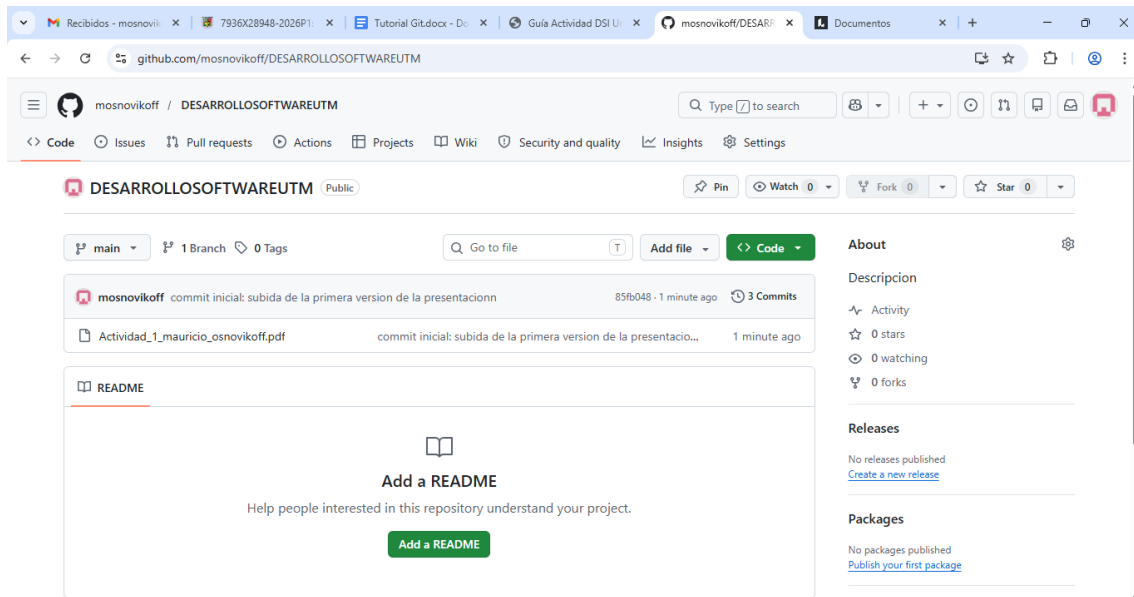
5. Configuración del Repositorio y Control de Versiones

Para el desarrollo del sistema de gestión de incidentes se configuró un repositorio público en GitHub utilizando Git como sistema de control de versiones. Se realizó la instalación y configuración inicial de Git, estableciendo la identidad del usuario mediante comandos de configuración global.

Posteriormente, se creó un repositorio local para el proyecto, se realizó el commit inicial y se vinculó el proyecto con un repositorio remoto en GitHub. Esto permite mantener un

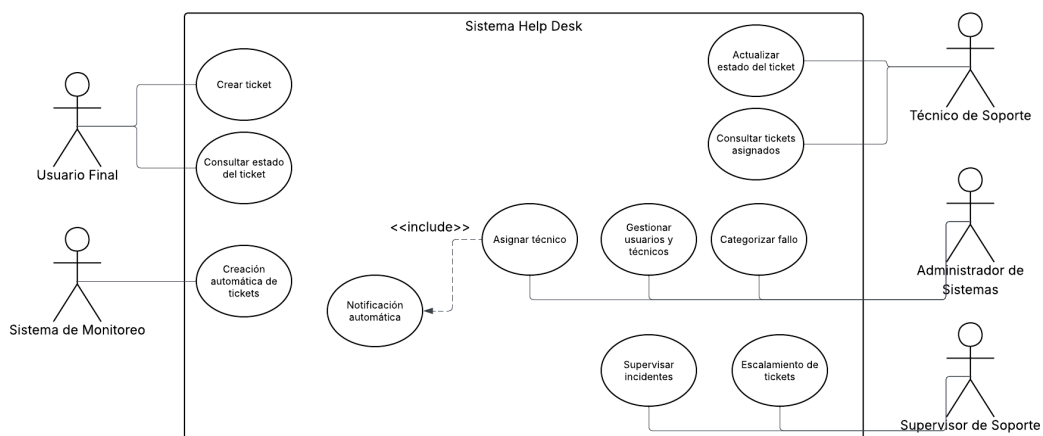


historial de cambios y facilitar la actualización continua del proyecto durante el desarrollo de la actividad.



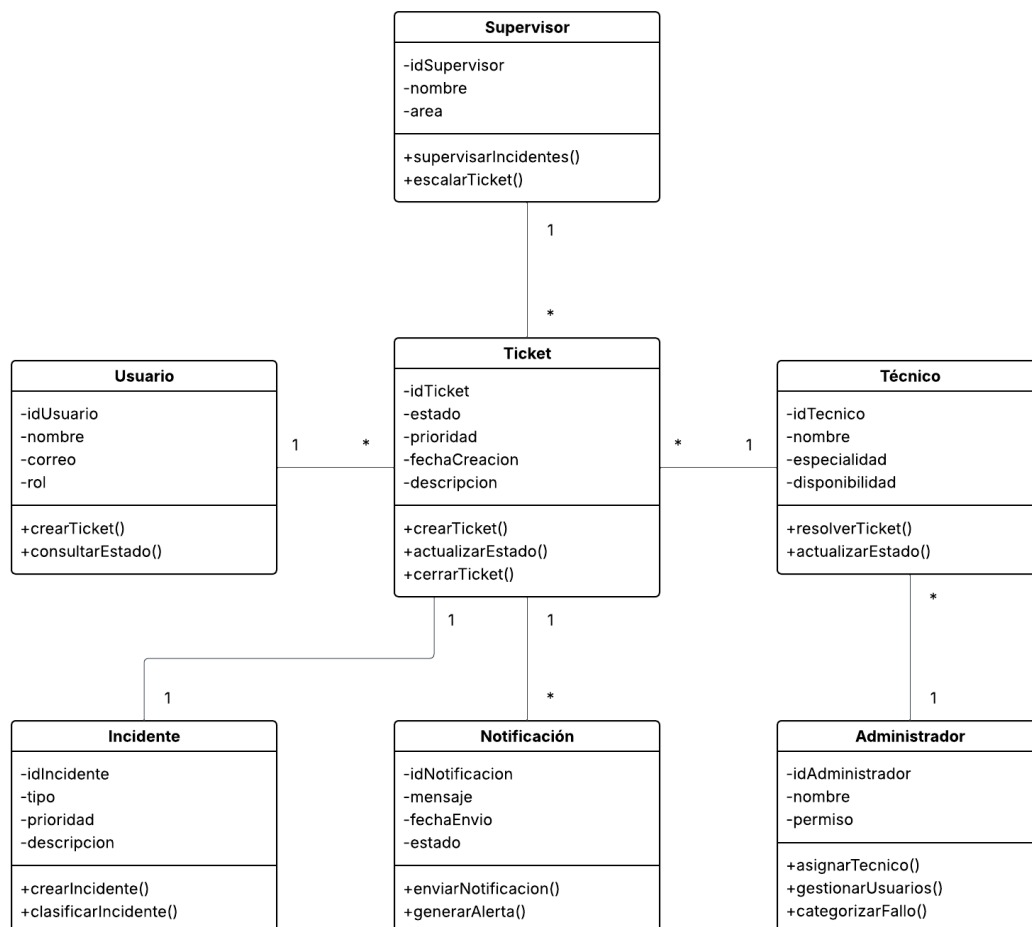
6. Diagrama de Casos de Uso

El siguiente diagrama de casos de uso representa las interacciones entre los actores principales y las funcionalidades del sistema Help Desk. Se representan las principales funcionalidades relacionadas con la creación, gestión, seguimiento y escalamiento de tickets. También se incorpora la relación `<<include>>` entre la asignación de técnicos y las notificaciones automáticas, indicando que el sistema genera avisos automáticamente cuando un incidente es asignado.



7. Diagrama de Clases UML

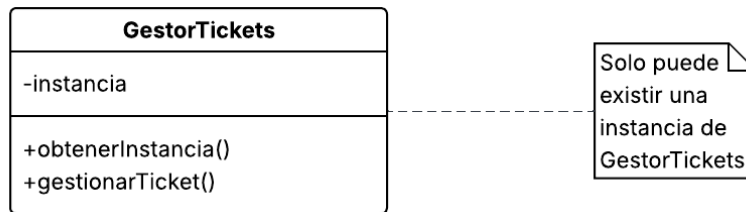
El diagrama de clases UML representa la estructura interna del sistema Help Desk, mostrando las principales clases, atributos, métodos y relaciones entre los objetos que conforman la solución. Las clases principales del sistema incluyen Ticket, Usuario, Técnico, Administrador, Supervisor, Incidente y Notificación. Cada una representa una entidad importante dentro del proceso de gestión de incidentes en el Data Center. Las relaciones entre clases permiten modelar la interacción entre usuarios, técnicos y tickets, así como los procesos de notificación y supervisión de incidentes.



8. Aplicación de Patrones de Diseño

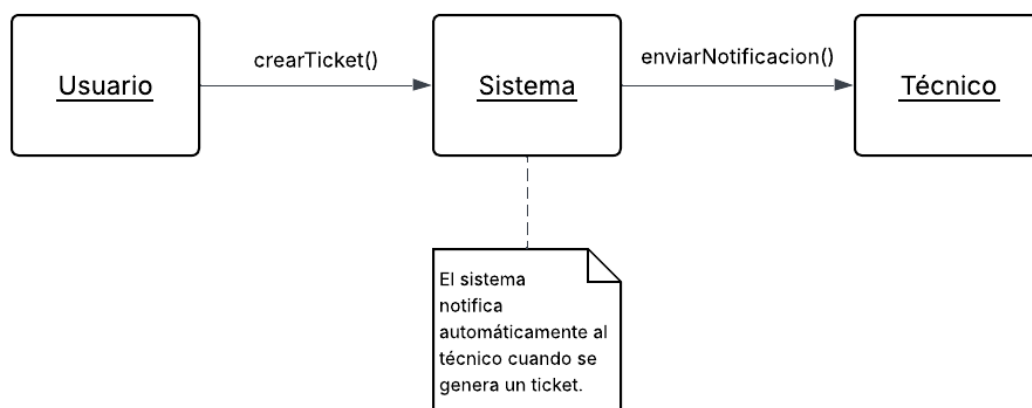
Singleton: Se utilizará para el gestor central de tickets del sistema Help Desk. Este patrón permite garantizar que exista una única instancia encargada de administrar la creación, actualización y seguimiento de los tickets dentro de la plataforma. La utilización de Singleton facilita el control centralizado de los incidentes y evita inconsistencias provocadas por múltiples instancias administrando la misma

información. Además, contribuye a mantener una estructura organizada y eficiente en el sistema.



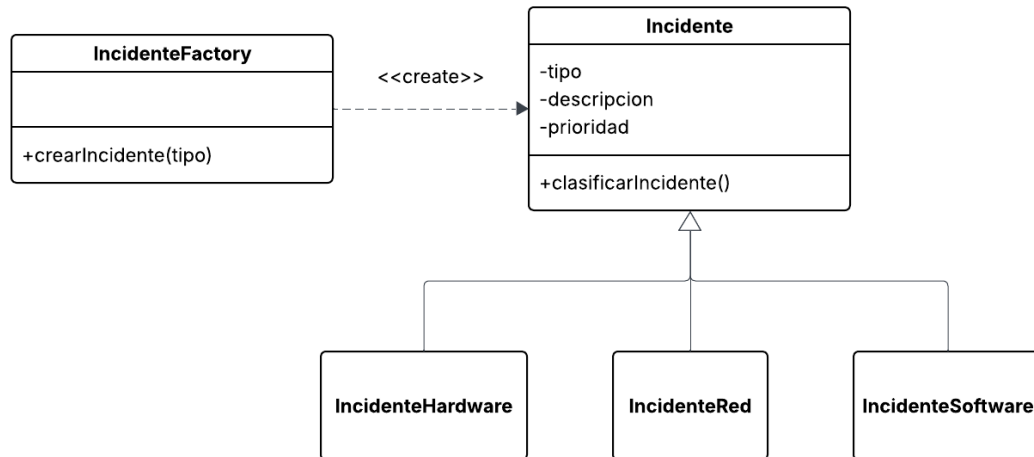
Implementación del patrón Singleton.

Observer: Se utilizará para el sistema de notificaciones automáticas del Help Desk. Este patrón permite que los técnicos reciban alertas cuando se genere o asigne un nuevo incidente dentro de la plataforma. Mediante este mecanismo, el sistema puede informar automáticamente los cambios relacionados con los tickets sin necesidad de que los usuarios consulten constantemente el estado de los incidentes. Esto mejora la comunicación y agiliza el proceso de atención y resolución de problemas.



Implementación del patrón Observer mediante diagrama de secuencia.

Factory Method: Se utilizará para la creación de diferentes tipos de incidentes dentro del sistema Help Desk. A través de este patrón, el sistema podrá generar objetos específicos según la categoría del fallo reportado, como incidentes de hardware, red o software. La implementación de este patrón facilita la escalabilidad del sistema, permitiendo agregar nuevos tipos de incidentes sin modificar la estructura principal de creación de objetos, también mejora la organización y reutilización del código.



Implementación del patrón Factory Method.

MVC (Model-View-Controller): Se utilizará para organizar la estructura general del sistema Help Desk. Este patrón divide la aplicación en tres componentes principales: modelo, vista y controlador. El modelo representa las clases y datos del sistema, como Ticket, Usuario e Incidente. La vista corresponde a las interfaces utilizadas por los usuarios y técnicos para interactuar con la plataforma. Finalmente, el controlador administra la lógica de negocio y coordina las acciones entre la vista y el modelo. La utilización de MVC permite desarrollar un sistema más organizado, mantenible y escalable, facilitando futuras mejoras y actualizaciones.



Implementación del patrón MVC.

9. Implementación de Gitflow y Control de Versiones

Con el objetivo de organizar el desarrollo del proyecto de manera estructurada, se implementó una estrategia básica de Gitflow utilizando diferentes ramas dentro del repositorio GitHub. La rama main se utilizó para almacenar la versión estable del proyecto, mientras que la rama develop permite integrar los avances generales realizados durante el desarrollo del sistema Help Desk. Adicionalmente, se creó la rama feature/documentacion-gitflow para trabajar de forma independiente en la documentación relacionada con el flujo de versiones y evidencias del proyecto. El uso de ramas y commits facilita el seguimiento de las modificaciones realizadas en el

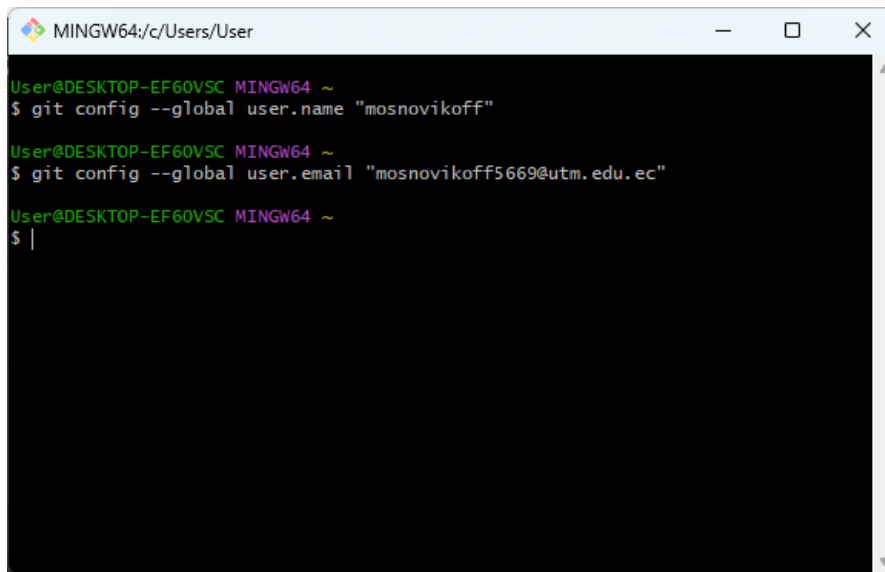


proyecto. GitHub permite visualizar el historial de cambios y mantener evidencia del flujo de trabajo aplicado durante la actividad.

Enlace Repositorio Github

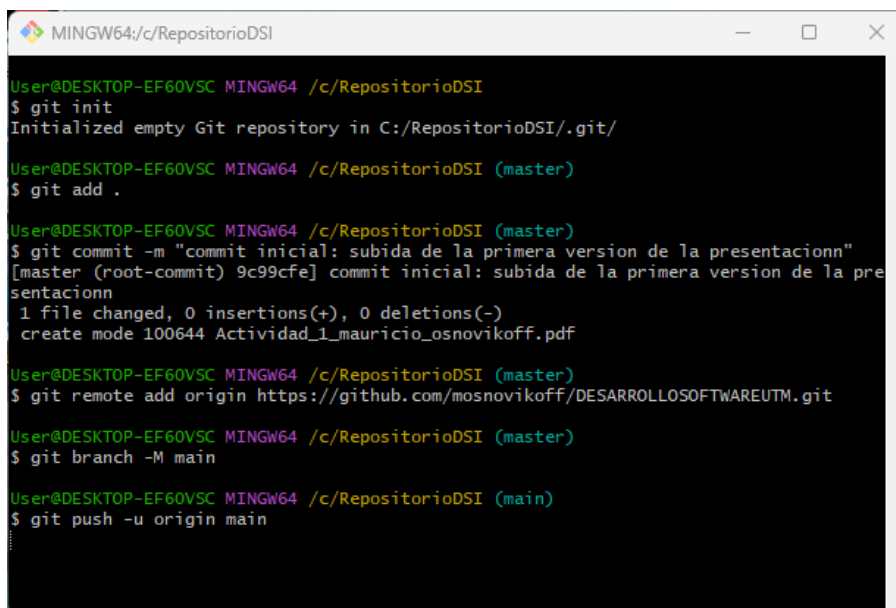
<https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM>

10. Capturas de Pantalla



```
MINGW64:/c/Users/User
User@DESKTOP-EF60VSC MINGW64 ~
$ git config --global user.name "mosnovikoff"
User@DESKTOP-EF60VSC MINGW64 ~
$ git config --global user.email "mosnovikoff5669@utm.edu.ec"
User@DESKTOP-EF60VSC MINGW64 ~
$ |
```

Configuración de identidad de usuario en Git



```
MINGW64:/c/RepositorioDSI
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI
$ git init
Initialized empty Git repository in C:/RepositorioDSI/.git/
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (master)
$ git add .
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (master)
$ git commit -m "commit inicial: subida de la primera version de la presentacionn"
[master (root-commit) 9c99cfe] commit inicial: subida de la primera version de la presentacionn
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Actividad_1_mauricio_osnovikoff.pdf
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (master)
$ git remote add origin https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM.git
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (master)
$ git branch -M main
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (main)
$ git push -u origin main
```

Creación y publicación inicial del repositorio en GitHub



```
MINGW64/c/RepositorioDSI
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM.git
   8f93930..85fb048  main -> main
branch 'main' set up to track 'origin/main'.

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (main)
$ git add .

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (main)
$ git commit -m "actualizacion: se agrego diagramas de uso y de clases"
[main cb7bb20] actualizacion: se agrego diagramas de uso y de clases
1 file changed, 0 insertions(+), 0 deletions(-)

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (main)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 272.48 KiB | 11.85 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM.git
   85fb048..cb7bb20  main -> main

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (main)
$
```

Registro de actualización de diagramas UML mediante commit

```
MINGW64/c/RepositorioDSI
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (main)
$ git checkout -b develop
Switched to a new branch 'develop'

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (develop)
$ git branch
* develop
  main

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (develop)
$ git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM/pull/new/develop
remote:
To https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM.git
 * [new branch]   develop -> develop
branch 'develop' set up to track 'origin/develop'.

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (develop)
$ |
```

Creación de la rama develop



```
MINGW64:/c/RepositorioDSI
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (develop)
$ git branch
* develop
main

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (develop)
$ git checkout -b feature/documentacion-gitflow
Switched to a new branch 'feature/documentacion-gitflow'

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ git branch
* feature/documentacion-gitflow
main

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ git push -u origin feature/documentacion-gitflow
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/documentacion-gitflow' on GitHub by visiting:
remote:   https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM/pull/new/feature/documentacion-gitflow
remote:
To https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM.git
 * [new branch]   feature/documentacion-gitflow -> feature/documentacion-gitflow
branch 'feature/documentacion-gitflow' set up to track 'origin/feature/documentacion-gitflow'.

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ |
```

Creación de la rama feature/documentacion-gitflow

```
MINGW64:/c/RepositorioDSI
User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ git status
On branch feature/documentacion-gitflow
Your branch is up to date with 'origin/feature/documentacion-gitflow'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Actividad_1_mauricio_osnovikoff.pdf

no changes added to commit (use "git add" and/or "git commit -a")

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ git add .

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ git commit -m "documentacion: se agrego implementacion de Gitflow"
[feature/documentacion-gitflow f579c4b] documentacion: se agrego implementacion de Gitflow
1 file changed, 0 insertions(+), 0 deletions(-)

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 52.22 KiB | 5.22 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/mosnovikoff/DESARROLLOSOFTWAREUTM.git
   95aa232..f579c4b  feature/documentacion-gitflow -> feature/documentacion-gitflow

User@DESKTOP-EF60VSC MINGW64 /c/RepositorioDSI (feature/documentacion-gitflow)
$ |
```

Commit realizado en la rama feature/documentacion-gitflow



11. Conclusiones

El desarrollo de la actividad permitió comprender la importancia del análisis y modelado de sistemas mediante diagramas UML, facilitando la representación de actores, funcionalidades, clases y procesos relacionados con la gestión de incidentes en un entorno de Data Center.

La aplicación de patrones de diseño como Singleton, Observer, Factory Method y MVC permitió identificar buenas prácticas orientadas a la organización, reutilización y mantenibilidad del software, mejorando la estructura lógica del sistema Help Desk.

El uso de Git, GitHub y la estrategia básica de Gitflow facilitó el control de versiones y el seguimiento de cambios realizados durante el proyecto, permitiendo trabajar de manera organizada y simulando un entorno profesional de desarrollo de software.